同济大学

本科生毕业设计（论文）

外文科技文献译文

译文题目	AutoGen：通过多智能体对话赋能下一代大语言模型应用
（外文题目）	AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation
学　　院	计算机科学与技术学院
专　　业	软件工程
学　　号	2250758
学生姓名	林继申
日　　期	2026 年 5 月 22 日

指导教师签名 黄志 日期 2026 年 5 月 22 日

AutoGen：通过多智能体对话赋能下一代大语言模型应用

Qingyun Wu，Gagan Bansal，Jieyu Zhang，Yiran Wu，Beibin Li
Erkang Zhu，Li Jiang，Xiaoyun Zhang，Shaokun Zhang，Jiale Liu
Ahmed Awadallah，Ryen W. White，Doug Burger，Chi Wang[1]
微软研究院，宾夕法尼亚州立大学
华盛顿大学，西安电子科技大学

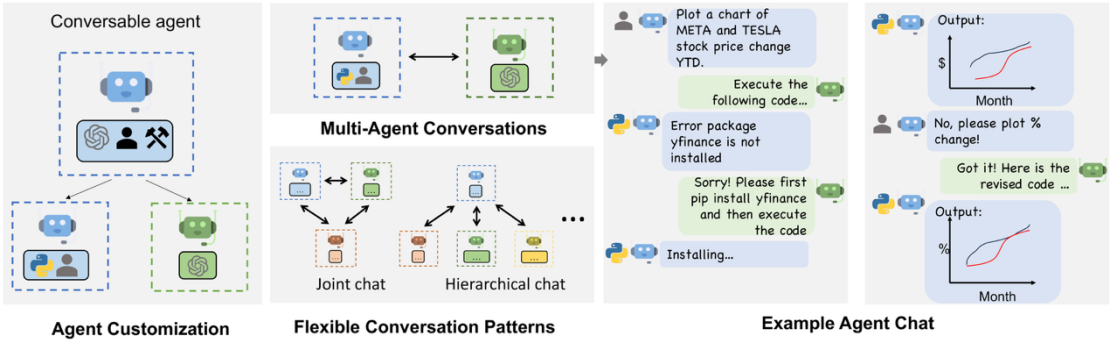


图 1：AutoGen 通过多智能体对话支持多样化的基于 LLM 的应用。（左）AutoGen 智能体具备可对话性与可定制性，并且可以基于 LLM、工具、人类，或这些能力的组合来构建。（上中）智能体可以通过对话来解决任务。（右）智能体可以形成聊天过程，其中也可以包含人类参与环节。（下中）该框架支持灵活的对话模式。

摘要

AutoGen[2]是一个开源框架，使开发者能够通过多个可以相互对话以完成任务的智能体来构建 LLM 应用。AutoGen 智能体具备可定制性和可对话性，并且能够在多种模式下运行，这些模式可以组合使用 LLM、人类输入和工具。借助 AutoGen，开发者还可以灵活定义智能体之间的交互行为。自然语言和计算机代码都可以用于为不同应用编排灵活的对话模式。AutoGen 可作为一个通用框架，用于构建复杂度和 LLM 能力需求各异的多类应用。实证研究表明，该框架在许多示例应用中具有有效性，覆盖领域包括数学、编程、问答、运筹优化、在线决策、娱乐等。

1 引言

大型语言模型（LLM）正在成为构建强大智能体的关键基础模块。这些智能体在许多现实任务中利用 LLM 进行推理、使用工具，并适应新的观察结果。随着可受益于 LLM 的任务范围不断扩展，任务复杂度也持续提高，一种直观的增强智能体能力的方式是使用多个智

[1] 通讯作者邮箱：auto-gen@outlook.com
[2] https://github.com/microsoft/autogen

能体进行协作。已有研究表明，多个智能体有助于促进发散性思维，提高事实性与推理能力，并提供验证机制。基于这一直觉以及早期证据所显示出的潜力，一个值得关注的问题是：如何基于多智能体方法，促进能够覆盖广泛领域和复杂度范围的 LLM 应用开发？

我们的核心洞见是使用多智能体对话来实现这一目标。得益于 LLM 的最新进展，至少有三个理由可以证明这一思路具有普遍可行性和实用价值。首先，由于面向聊天优化的 LLM（例如 GPT-4）表现出整合反馈的能力，LLM 智能体可以通过彼此之间或与人类之间的对话进行协作，例如在对话中提供和寻求推理、观察、批评与验证。其次，由于单个 LLM 可以呈现出广泛能力，尤其是在使用正确提示和推理设置进行配置时，不同配置的智能体之间的对话可以帮助以模块化且互补的方式组合这些广泛的 LLM 能力。第三，LLM 已经展示出在将复杂任务拆解为更简单子任务后解决复杂任务的能力。多智能体对话可以以一种直观方式支持这种划分与整合。如何利用上述洞见，并支持不同应用在协调多个智能体方面的共同需求？这些智能体可能由 LLM、人类或工具支撑，并且呈现出不同能力。我们需要一个多智能体对话框架，其抽象应足够通用，实现应足够有效，同时具备满足不同应用需求的灵活性。实现这一目标需要回答两个关键问题：(1) 如何设计能够在多智能体协作中发挥作用、可复用、可定制且有效的单个智能体？(2) 如何开发一个直接而统一的接口，以容纳范围广泛的智能体对话模式？在实践中，复杂度不同的应用可能需要具有特定能力的不同智能体集合，也可能需要不同的对话模式，例如单轮或多轮对话、不同的人类参与模式，以及静态或动态对话。此外，开发者可能希望能够灵活地用自然语言或代码来编排智能体交互。若不能充分解决这两个问题，框架的适用范围和通用性将受到限制。

尽管同时期已有对多智能体方法的探索[3]，本文提出 AutoGen，一个通用化的多智能体对话框架（图 1），它基于以下新概念。

1. 可定制且可对话的智能体。AutoGen 采用通用的智能体设计，使智能体能够利用 LLM、人类输入、工具，或它们的组合。其结果是，开发者可以通过选择并配置内置能力的子集，轻松且快速地创建具有不同角色的智能体，例如用于编写代码、执行代码、接入人类反馈、验证输出等的智能体。智能体后端也可以方便地扩展，以支持更多自定义行为。为使这些智能体适用于多智能体对话，每个智能体都被设计为可对话，即能够接收、响应并回复消息。在适当配置下，一个智能体可以自主地与其他智能体进行多轮对话，或在特定轮次请求人类输入，从而同时支持人类能动性和自动化。可对话智能体设计利用了最先进 LLM 通过聊天吸收反馈并推进任务的强大能力，同时也允许以模块化方式组合 LLM 的能力。

2. 对话编程。AutoGen 的一个基础洞见是，将复杂 LLM 应用工作流简化并统一为多智能体对话。因此，AutoGen 采用一种以智能体间对话为中心的编程范式。我们将这一范式称为对话编程，它通过两个主要步骤来简化复杂应用的开发：(1) 定义一组具有特定能力和角色的可对话智能体，如上所述；(2) 通过以对话为中心的计算和控制来编排智能体之间的交互行为。这两个步骤都可以通过自然语言与编程语言的融合来完成，从而构建具有广泛对话模式和智能体行为的应用。AutoGen 为这两个步骤提供了开箱即用的实现，并允许轻松扩展和实验。

AutoGen 还提供了一组使用可对话智能体和对话编程创建的多智能体应用。这些应用展示了 AutoGen 如何轻松支持复杂度各异、所需 LLM 能力不同的应用。此外，我们既在基准上进行了评估，也对新应用开展了试点研究。结果表明，AutoGen 可以帮助许多任务取得优

[3] 详见附录 A 中的详细讨论。

异性能，并在降低开发工作量的同时，支持使用 LLM 的创新方式。

2 AutoGen 框架

为了降低开发者在不同领域创建复杂 LLM 应用所需的工作量，AutoGen 的一项核心设计原则是使用多智能体对话来简化并整合多智能体工作流。该方法也旨在最大化已实现智能体的可复用性。本节介绍 AutoGen 的两个关键概念：可对话智能体和对话编程。

2.1 可对话智能体

在 AutoGen 中，可对话智能体是一个具有特定角色的实体，能够向其他可对话智能体传递消息，以发送和接收信息，例如发起或继续一段对话。它基于发送和接收的消息维护内部上下文，并可被配置为具备一组能力，例如由 LLM、工具或人类输入等实现的能力。智能体可以按照下一节所描述的编程行为模式采取行动。

由 LLM、人类和工具驱动的智能体能力。由于智能体的能力会直接影响其处理和回复消息的方式，AutoGen 允许灵活地为智能体赋予多种能力。AutoGen 支持智能体使用许多常见且可组合的能力，包括以下三类。**(1) LLM。**由 LLM 支撑的智能体利用先进 LLM 的多种能力，例如角色扮演、基于对话历史的隐式状态推断和任务推进、提供反馈、从反馈中适应，以及编写代码。这些能力可以通过新的提示技术[4]以不同方式组合，从而提升智能体的技能和自主性。AutoGen 还通过增强型 LLM 推理层提供更强的 LLM 推理特性，例如结果缓存、错误处理、消息模板等。**(2) 人类。**在许多 LLM 应用中，人类参与是理想的，甚至是必要的。AutoGen 允许人类通过由人类支撑的智能体参与智能体对话；这类智能体可根据配置，在对话的特定轮次请求人类输入。默认的用户代理智能体允许可配置的人类参与级别和模式，例如请求人类输入的频率与条件，也包括人类可选择跳过输入的选项。**(3) 工具。**由工具支撑的智能体具备通过代码执行或函数执行来调用工具的能力。例如，AutoGen 中默认的用户代理智能体能够执行 LLM 建议的代码，或发起 LLM 建议的函数调用。

智能体定制与协作。基于应用的具体需求，每个智能体都可以被配置为混合多种基础后端类型，从而在多智能体对话中表现出复杂行为。AutoGen 允许通过复用或扩展内置智能体，轻松创建具有专门能力和角色的智能体。图 2 中黄色阴影区域概述了 AutoGen 的内置智能体。ConversableAgent 类是最高层级的智能体抽象，默认情况下可以使用 LLM、人类和工具。AssistantAgent 和 UserProxyAgent 是两个预配置的 ConversableAgent 子类，分别代表常见的使用模式，即作为 AI 助手（由 LLM 支撑）和作为人类代理来请求人类输入或执行代码/函数调用（由人类和/或工具支撑）。

在图 1 右侧的示例中，一个由 LLM 支撑的助手智能体和一个由工具与人类支撑的用户代理智能体被共同部署来处理一项任务。这里，助手智能体借助 LLM 生成解决方案，并将解决方案传递给用户代理智能体。随后，用户代理智能体请求人类输入，或执行助手给出的代码，并将结果作为反馈传回助手。

通过允许可相互对话的自定义智能体，AutoGen 中的可对话智能体构成了有用的基础模块。然而，要开发能够使智能体在任务上取得有意义进展的应用，开发者还需要能够指定并塑造这些多智能体对话。

[4] 附录 C 给出了这类新型提示技术的一个示例，该技术使 AutoGen 中默认的 LLM 支持助手智能体能够在多步骤问题求解中与其他智能体进行对话。

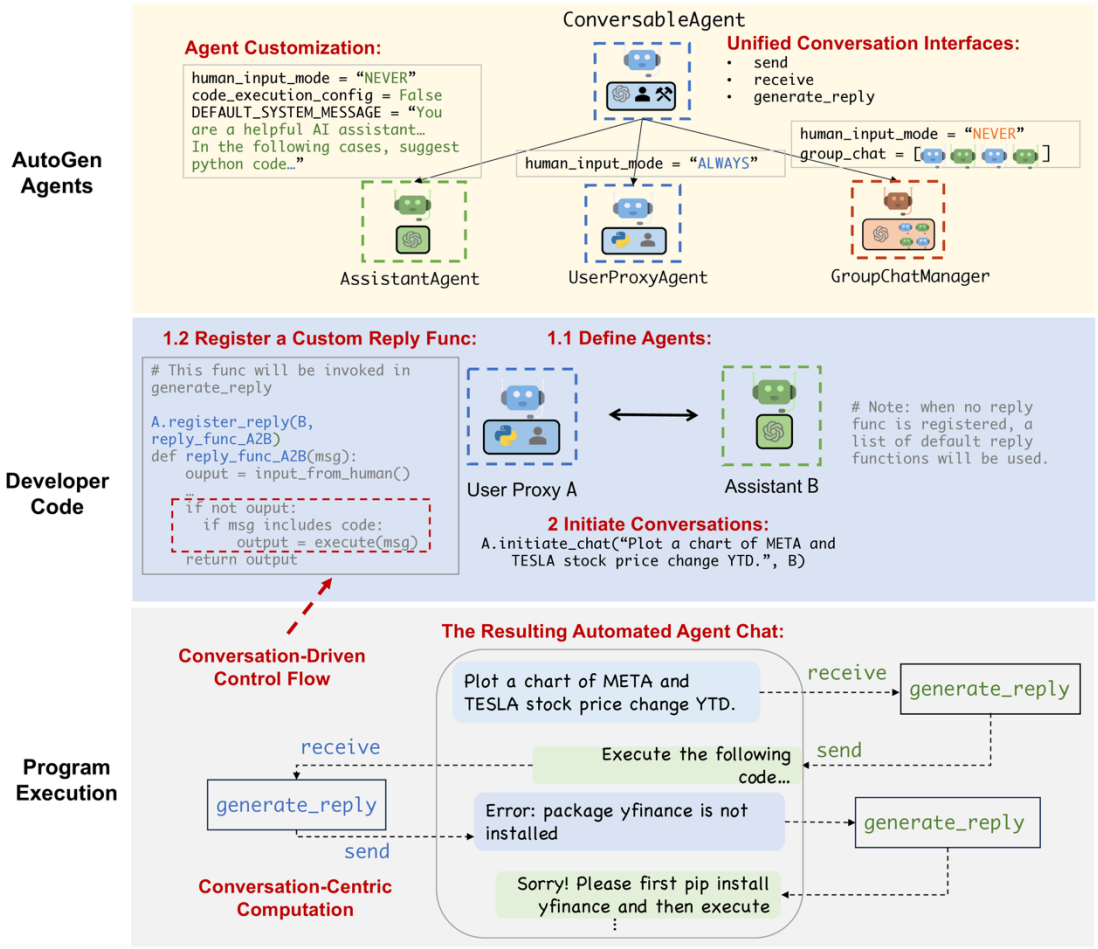


图 2：使用 AutoGen 编排多智能体对话的示意图。上部子图展示了 AutoGen 提供的内置智能体，这些智能体具有统一的对话接口并可被定制。中部子图展示了使用 AutoGen 开发一个带有自定义回复函数的双智能体系统的示例。下部子图展示了程序执行期间由该双智能体系统产生的自动化智能体聊天过程。

2.2 对话编程

作为上述问题的解决方案，AutoGen 使用对话编程这一范式。该范式考虑两个概念：第一个是计算，即智能体在多智能体对话中为计算其回复而采取的动作；第二个是控制流，即这些计算发生的顺序或条件。如应用部分将展示的，编排这些要素的能力有助于实现许多灵活的多智能体对话模式。在 AutoGen 中，这些计算以对话为中心。智能体采取与其所参与对话相关的动作，而这些动作会产生消息传递，进而触发后续对话，除非满足终止条件。类似地，控制流也由对话驱动。参与智能体关于向哪些智能体发送消息以及执行何种计算过程的决策，都是智能体间对话的函数。这一范式有助于将复杂工作流直观地理解为智能体采取行动以及智能体之间传递对话消息的过程。

图 2 给出了一个简单示意。下部子图展示了各个智能体如何执行与其角色相对应、以对话为中心的计算来生成回复，例如通过 LLM 推理调用和代码执行。任务通过对话框中显示的对话不断推进。中部子图展示了基于对话的控制流。当助手收到消息时，用户代理智能体通常会将人类输入作为回复发送。如果没有输入，它将转而执行助手消息中的任何代码。

AutoGen 具有以下设计模式，以促进对话编程：

1. 用于自动化智能体聊天的统一接口和自动回复机制。AutoGen 中的智能体具有统一

的对话接口，用于执行相应的以对话为中心的计算，其中包括用于发送/接收消息的 send/receive 函数，以及用于基于接收消息采取行动并生成回复的 generate_reply 函数。AutoGen 还引入并默认采用**智能体自动回复**机制，以实现由对话驱动的控制：一旦某个智能体收到来自另一智能体的消息，只要未满足终止条件，它就会自动调用 generate_reply 并将回复发送回发送方。AutoGen 提供了基于 LLM 推理、代码或函数执行、人类输入的内置回复函数。开发者也可以注册自定义回复函数，以定制智能体的行为模式，例如在回复发送方智能体之前先与另一个智能体聊天。在该机制下，一旦注册回复函数并初始化对话，对话流就会被自然诱导出来，因此智能体对话无需任何额外控制平面，即无需专门控制对话流的特殊模块，就可以自然推进。例如，使用图 2 蓝色阴影区域（标为“Developer Code”）中的开发者代码，即可方便地触发智能体之间的对话；该对话将自动进行，如图 2 灰色阴影区域（标为“Program Execution”）中的对话框所示。自动回复机制提供了一种去中心化、模块化且统一的方式来定义工作流。

2. 通过编程语言与自然语言融合进行控制。AutoGen 允许在多种控制流管理模式中使用编程语言和自然语言。**(1) 通过 LLM 进行自然语言控制。**在 AutoGen 中，可以通过用自然语言提示由 LLM 支撑的智能体来控制对话流。例如，AutoGen 中内置 AssistantAgent 的默认系统消息使用自然语言指示智能体：如果上一轮结果显示存在错误，则修复错误并重新生成代码。它还引导智能体将 LLM 输出限定在特定结构中，使其他由工具支撑的智能体更容易消费这些输出。例如，指示智能体在所有任务完成后回复“TERMINATE”，以终止程序。**(2) 编程语言控制。**在 AutoGen 中，可以使用 Python 代码来指定终止条件、人类输入模式和工具执行逻辑，例如最大自动回复次数。也可以注册经过编程的自动回复函数，使用 Python 代码控制对话流，如图 2 中标识为“Conversation-Driven Control Flow”的代码块所示。**(3) 自然语言与编程语言之间的控制转换。**AutoGen 还支持在自然语言和编程语言之间进行灵活的控制转换。开发者可以在自定义回复函数中调用包含某种控制逻辑的 LLM 推理，从而实现从代码到自然语言控制的转换；也可以通过 LLM 提出的函数调用，实现从自然语言到代码控制的转换。

在对话编程范式中，可以实现多种模式的多智能体对话。除具有预定义流程的静态对话外，AutoGen 还支持涉及多个智能体的动态对话流。AutoGen 提供两种通用方式来实现这一点。(1) 定义 generate_reply 函数：在自定义 generate_reply 函数内部，一个智能体可以在保持当前对话的同时，根据当前消息和上下文的内容调用与其他智能体的对话。(2) 函数调用：在该方法中，LLM 根据对话状态决定是否调用某个特定函数。通过在被调用函数中向其他智能体发送消息，LLM 可以驱动动态的多智能体对话。此外，AutoGen 还通过内置的 GroupChatManager 支持更复杂的动态群聊。它可以动态选择下一位发言者，然后将其回复广播给其他智能体。我们将在第 3 节 A5 中进一步阐述这一特性及其应用。我们提供了已实现且可运行的系统，以展示所有这些不同模式，其中部分模式在图 3 中进行了可视化。

3 AutoGen 的应用

我们展示了六个使用 AutoGen 构建的应用（见图 3），以说明其在简化高性能多智能体应用开发方面的潜力。这些应用基于其现实相关性（A1、A2、A4、A5、A6）、问题难度和 AutoGen 所支持的求解能力（A1、A2、A3、A4），以及创新潜力（A5、A6）进行选择。这些标准共同展示了 AutoGen 在推进 LLM 应用生态方面的作用。

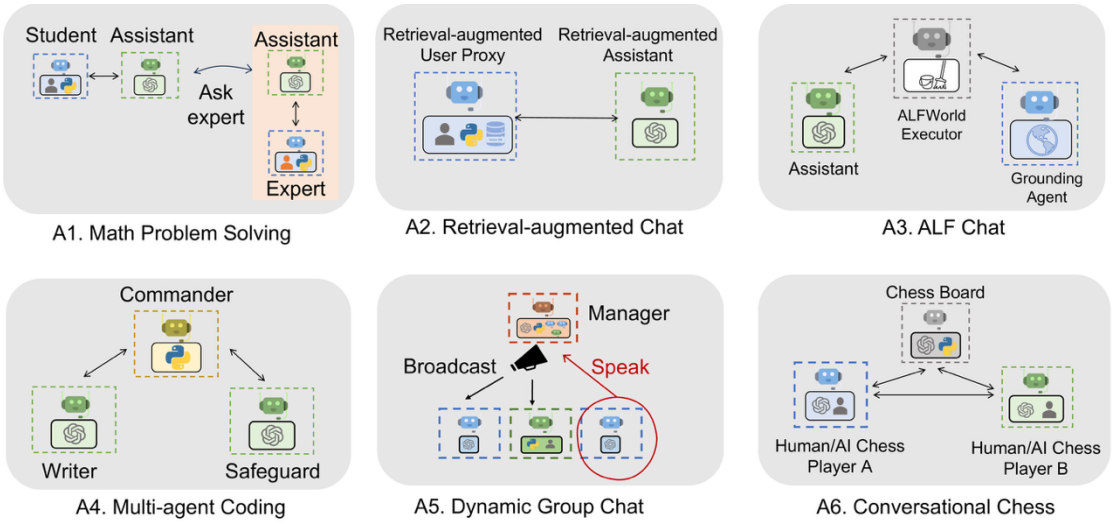


图 3：使用 AutoGen 构建的六个多样化应用示例。它们的对话模式展示了 AutoGen 的灵活性和能力。

A1：数学问题求解

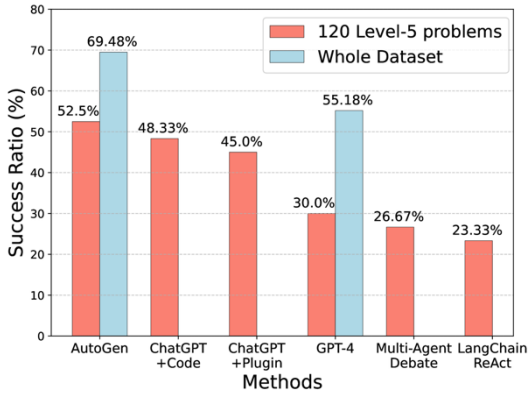
数学是一门基础学科，利用 LLM 辅助数学问题求解的前景开启了大量新的应用和探索方向，包括个性化 AI 辅导、AI 研究助手等。本节展示 AutoGen 如何帮助开发用于数学问题求解的 LLM 应用，并体现其在支持多种问题求解范式方面的强性能与灵活性。

（场景 1）我们能够通过直接复用 AutoGen 中两个内置智能体，构建一个自主数学问题求解系统。我们在 MATH 数据集上评估了该系统和若干替代方法，包括 Multi-Agent Debate、LangChain ReAct、原始 GPT-4 等开源方法，以及 ChatGPT + Code Interpreter 和 ChatGPT + Plugin（Wolfram Alpha）等商业产品，并在图 4a 中汇总结果。我们在随机选取的 120 个 5 级问题以及整个[5]MATH 测试数据集上进行了评估。结果表明，与替代方法相比，AutoGen 的内置智能体即使开箱即用也已经产生了更好的性能，甚至优于这些商业方案。**（场景 2）**我们还展示了在 AutoGen 帮助下实现人类参与的问题求解过程。若要在 AutoGen 中纳入人类反馈，只需在场景 1 系统的 UserProxyAgent 中设置 human_input_mode='ALWAYS'。我们展示了该系统能够有效整合人类输入，从而解决没有人类参与时无法解决的挑战性问题。**（场景 3）**我们进一步展示了一种新场景，其中多个人类用户可以在问题求解过程中参与对话。针对这些场景的实验和案例研究表明，与我们实验过的其他解决方案相比，AutoGen 能够带来更好的性能或新的体验。由于篇幅限制，评估细节以及三个场景中的案例研究见附录。

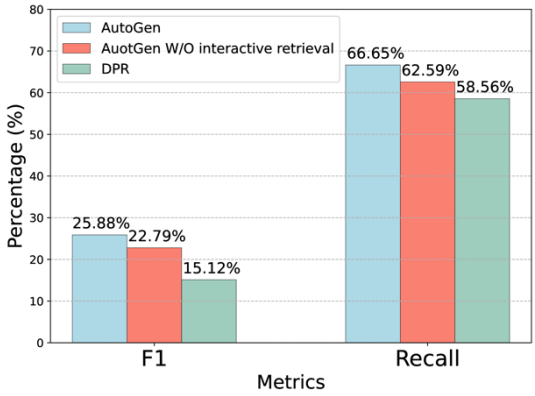
A2：检索增强的代码生成与问答

通过引入外部文档，检索增强已成为缓解 LLM 内在局限的一种实用且有效的方法。在本节中，我们使用 AutoGen 构建了一个名为 Retrieval-augmented Chat 的检索增强生成（RAG）系统。该系统由两个智能体组成：检索增强用户代理智能体和检索增强助手智能体，二者都从 AutoGen 的内置智能体扩展而来。检索增强用户代理包含一个向量数据库，并使用 SentenceTransformers 作为上下文检索器。Retrieval-augmented Chat 的详细工作流说明见附录。

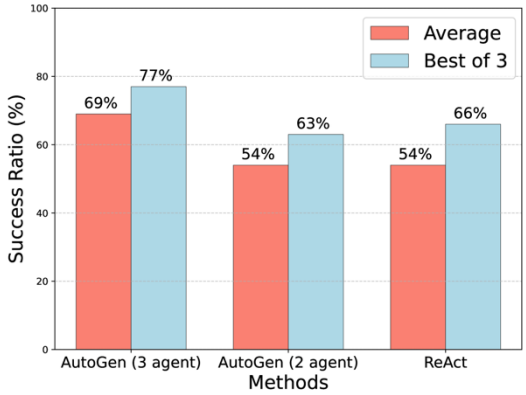
[5] 我们未在整个数据集上评估 ChatGPT，因为这需要大量人工工作，并且受其每小时消息数量限制。Multi-agent debate 和 LangChain ReAct 也未被评估，因为它们在较小测试集上的表现低于原始 GPT-4。



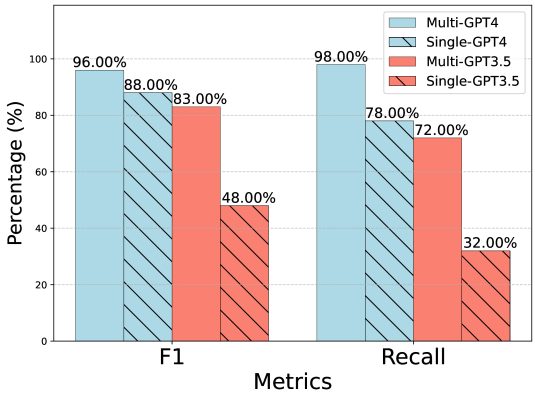
(a) A1：MATH 上的性能（使用 GPT-4）



(b) A2：问答任务（使用 GPT-3.5）。



(c) A3：ALFWorld 上的性能。



(d) A4：OptiGuide 上的性能。

图 4：A1 到 A4 四个应用上的性能。子图 a 表明，AutoGen 智能体可以开箱即用，在数学问题求解任务上达到最具竞争力的性能；子图 b 表明，AutoGen 可用于实现有效的检索增强，并实现一种新的交互式检索特性，从而提升问答任务性能；子图 c 表明，AutoGen 可用于引入带有落地智能体的三智能体系统，以提升 ALFWorld 上的性能；子图 d 表明，在需要安全防护的编程任务中，多智能体设计有助于提升性能。

我们在问答和代码生成两个场景中评估 Retrieval-augmented Chat。**（场景 1）**我们首先在 Natural Questions 数据集上对自然问答进行评估，并在图 4b 中报告结果。在该评估中，我们遵循既有评估实践[6]，将系统与 DPR（Dense Passage Retrieval，密集段落检索）进行比较。借助对话式设计和自然语言控制，AutoGen 在该应用中引入了一种新的交互式检索特性：每当检索到的上下文不包含所需信息时，基于 LLM 的助手不会终止，而会回复“Sorry, I cannot find any information about... UPDATE CONTEXT.”，这将触发更多检索尝试。我们开展了一项消融研究：当未找到相关信息时，提示助手智能体说“I don't know”，而不是“UPDATE CONTEXT.”，并在图 4b 中报告结果。结果表明，交互式检索机制在过程中确实发挥了不可忽视的作用。我们在附录中给出了使用这一有吸引力特性的具体示例和结果。**（场景 2）**我们进一步展示了 Retrieval-augmented Chat 如何基于给定代码库辅助生成代码，该代码库包含 GPT-4 训练数据中未包含的代码。两个场景的评估和演示细节见附录。

A3：文本世界环境中的决策

在本小节中，我们展示 AutoGen 如何用于开发涉及交互式或在线决策的有效应用。我

[6] 图 4b 中 DPR 与 GPT-3.5 结合的结果来自 Adlakha 等人（2023）。本文使用 GPT-3.5 作为 GPT-3.5-turbo 的简称。

们使用 ALFWorld 基准开展研究，该基准包含一组多样化的、基于语言的合成家庭环境交互式决策任务。

借助 AutoGen，我们实现了一个双智能体系统来求解 ALFWorld 任务。该系统由一个由 LLM 支撑的助手智能体和一个执行器智能体组成；前者负责提出完成任务的计划，后者负责在 ALFWorld 环境中执行动作。该系统集成了 ReAct 提示，并能够取得相近性能。ReAct 和基于 AutoGen 的双智能体系统共同面临的一个常见挑战是，它们偶尔无法利用关于物理世界的基本常识知识。这一缺陷可能导致系统因重复错误而陷入循环。幸运的是，AutoGen 的模块化设计使我们能够有效解决这一问题。借助 AutoGen，我们能够引入一个落地智能体，该智能体在系统出现重复错误的早期迹象时提供关键常识知识，例如“你必须先找到并拿起该物体，才能检查它。你必须先前往目标物体所在的位置，才能使用它。”这显著增强了系统避免陷入错误循环的能力。我们比较了系统的两个变体与 GPT-3.5-turbo 和 ReAct[7]在 ALFWorld 的 134 个未见任务上的任务求解性能，并在图 4c 中报告结果。结果表明，引入落地智能体平均可带来 15%的性能增益。通过检查系统输出，我们观察到，落地智能体通过在适当时机提供背景常识知识，显著降低了系统坚持错误计划的倾向，从而避免了错误循环的产生。系统比较的示例轨迹见附录图。

A4：多智能体编程

在本小节中，我们使用 AutoGen 构建了一个基于 OptiGuide 的多智能体编程系统。OptiGuide 是一个擅长编写代码来解释优化解并回答用户问题的系统，例如探索改变某项供应链决策的影响，或理解优化器为何做出某一特定选择。图 3 的第二个子图展示了基于 AutoGen 的实现。工作流如下：终端用户向 Commander 智能体发送问题，例如“如果我们禁止从供应商 1 向烘焙厂 2 发货，会怎样？”Commander 与两个助手智能体协同回答该问题，这两个智能体包括 Writer 和 Safeguard。Writer 会编写代码并将代码发送给 Commander。收到代码后，Commander 会与 Safeguard 一起检查代码安全性；如果通过检查，Commander 将使用外部工具（例如 Python）执行代码，并请求 Writer 解释执行结果。例如，Writer 可能会说：“如果我们禁止从供应商 1 向烘焙厂 2 发货，总成本将增加 10.5%。”随后，Commander 将这一结论性回答提供给终端用户。如果在某一步出现异常，例如 Safeguard 提出安全红旗，Commander 会将问题连同调试信息重新转交给 Writer。该过程可能重复多次，直到用户问题得到回答或超时。

借助 AutoGen，OptiGuide 的核心工作流代码从 430 多行减少到 100 行，从而显著提升了生产力。我们在附录中提供了 ChatGPT + Code Interpreter 与基于 AutoGen 的 OptiGuide 在用户体验方面的详细比较，其中显示基于 AutoGen 的 OptiGuide 可以节省约 3 倍的用户时间，并平均减少 3 到 5 倍的用户交互。我们还进行了消融实验，表明多智能体抽象是必要的。具体而言，我们构建了一种单智能体方法，由单个智能体同时执行代码编写和安全防护流程。我们在一个包含 100 个编程任务的数据集上测试了单智能体和多智能体方法，该数据集被构造为包含数量相等的安全任务和不安全任务。图 4d 报告的评估结果显示，多智能体设计在识别不安全代码方面将 F1 分数提升了 8%（使用 GPT-4）和 35%（使用 GPT-3.5-turbo）。

[7] ReAct 的结果是通过直接运行其官方代码并采用默认设置得到的。该代码使用 text-davinci-003 作为后端语言模型，且不支持 GPT-3.5-turbo 或 GPT-4。

A5：动态群聊

AutoGen 原生支持动态群聊通信模式。在该模式中，参与智能体共享同一上下文，并以动态方式与其他智能体对话，而不是遵循预定义顺序。动态群聊依赖正在进行的对话来引导智能体之间的交互流。这些特性使动态群聊非常适合那些不需要严格通信顺序即可受益于协作的场景。在 AutoGen 中，GroupChatManager 类充当智能体间对话的指挥者，并重复执行以下三个步骤：动态选择发言者、收集所选发言者的回复，以及广播该消息（图 3-A5）。对于动态发言者选择组件，我们使用角色扮演式提示。通过在 12 个手工构造的复杂任务上进行试点研究，我们观察到，与纯粹基于任务的提示相比，使用角色扮演提示通常会使系统在问题求解和发言者选择过程中更有效地同时考虑对话上下文和角色对齐。因此，这会带来更高的成功率和更少的 LLM 调用。详细结果见附录。

A6：对话式国际象棋

使用 AutoGen，我们开发了 Conversational Chess，这是一个自然语言接口游戏，如图 3 最后一个子图所示。它为棋手提供内置智能体，棋手可以是人类或 LLM；同时还提供一个第三方棋盘智能体，用于基于标准规则提供信息并验证走法。

借助 AutoGen，我们实现了两个关键特性。第一，自然、灵活且富有吸引力的游戏动态，这由 AutoGen 中可定制的智能体设计支持。Conversational Chess 支持一系列游戏模式，包括 AI-AI、AI-人类和人类-人类，并且可以在单局游戏过程中在这些模式之间无缝切换。附录图中给出了这些富有娱乐性的游戏动态示例。第二，落地性，这是维护游戏完整性的关键方面。在游戏过程中，棋盘智能体会检查每个提议走法的合法性；如果某个走法无效，该智能体会返回错误，提示棋手智能体在继续之前重新提出合法走法。该过程确保只执行有效走法，并有助于维持一致的游戏体验。作为一项消融研究，我们移除了棋盘智能体，改为仅依赖相关提示“you should make sure both you and the opponent are making legal moves”来约束其走法。结果表明，在没有棋盘智能体的情况下，非法走法会导致游戏中断。该模块化设计提供了灵活性，使系统能够随着游戏规则演化或不同国际象棋规则变体的变化，快速调整棋盘智能体。该消融研究的完整演示见附录。

4 讨论

本文介绍了一个开源库 AutoGen，它融合了可对话智能体和对话编程范式。该库使用适合多智能体协作的高能力智能体。它在智能体之间提供统一的对话接口，并配备自动回复机制，有助于建立一种智能体交互接口：既能利用面向聊天优化且具有广泛能力的 LLM 的优势，又能适配范围广泛的应用。AutoGen 是一个通用框架，可用于创建和实验多智能体系统，并能够轻松满足多种实践需求，例如复用、定制和扩展已有智能体，以及编排它们之间的对话。

第 3 节详述的实验表明，该方法带来了诸多收益。采用 AutoGen 后，已有应用获得了性能提升（超过当前最先进方法）、开发代码减少以及人工负担降低等结果。AutoGen 为开发者提供了灵活性，正如 A1（场景 3）、A5 和 A6 所展示的，AutoGen 使多智能体聊天能够遵循动态模式，而不是固定的来回交互。它允许人类以对话方式与多个 AI 智能体共同参与活动。尽管这些应用较为复杂，其中大多数涉及两个以上智能体或动态的多轮智能体协作，

但基于 AutoGen 的实现仍然直接明了。将任务划分给不同智能体可以促进模块化。此外，由于每个智能体都可以被独立开发、测试和维护，该方法简化了整体开发和代码管理。

尽管这项工作仍处于早期实验阶段，它为众多未来方向和研究机会铺平了道路。例如，我们可以探索如何将现有智能体实现有效集成到我们的多智能体框架中，并研究多智能体工作流中自动化与人类控制之间的最优平衡。随着我们进一步开发并完善 AutoGen，我们旨在研究哪些策略，例如智能体拓扑和对话模式，能够在优化整体效率等因素的同时，带来最有效的多智能体对话。虽然增加智能体数量以及其他自由度为解决更复杂问题提供了机会，但也可能引入新的安全挑战，需要进一步研究和审慎考虑。

我们在附录中提供了更多讨论，包括 AutoGen 使用指南和未来工作方向。我们希望 AutoGen 能够在开发速度、实验便利性以及整体有效性和安全性方面帮助改进许多 LLM 应用。我们积极欢迎更广泛社区的贡献。

伦理声明

AutoGen 框架的开发和使用可能引发若干潜在伦理考量。

(1) 隐私与数据保护：该框架允许人类参与智能体之间的对话。必须确保用户数据和对话受到保护，并确保开发者采取适当措施维护隐私。

(2) 偏见与公平性：已有研究表明，LLM 会表现出其训练数据中存在的偏见。在 AutoGen 框架中使用 LLM 时，关键在于处理并缓解智能体间对话中可能出现的任何偏见。开发者应意识到潜在偏见，并采取措施确保公平性和包容性。

(3) 问责与透明性：正如未来工作部分所讨论的，由于该框架涉及多个智能体的对话与协作，建立清晰的问责和透明机制十分重要。用户应能够理解并追踪相关智能体的决策过程，以确保问责，并处理任何潜在问题或偏见。

(4) 信任与依赖：AutoGen 利用人类理解与智能，同时通过智能体之间的对话提供自动化。必须考虑这种交互对用户体验、信任以及对 AI 系统依赖的影响。清晰沟通并教育用户理解系统能力与局限将至关重要。

(5) 非预期后果：如前文所述，在复杂任务中使用多智能体对话和自动化可能产生非预期后果。特别是允许 LLM 智能体通过代码执行或函数调用改变外部环境，例如安装软件包可能存在风险。开发者应仔细考虑风险，并确保设置适当防护措施，以防止损害或负面结果。

致谢

本报告所呈现的工作得益于与 Peter Lee、Johannes Gehrke、Eric Horvitz、Steven Lucco、Umesh Madan、Robin Moeur、Piali Choudhury、Saleema Amershi、Adam Fourney、Victor Dibia、Guoqing Zheng、Corby Rosset、Ricky Loynd、Ece Kamar、Rafah Hosn、John Langford、Ida Momennejad、Brian Krabach、Taylor Webb、Shanka Subhra Mondal、Wei-ge Chen、Robert Gruen、Yinan Li、Yue Wang、Suman Nath、Tanakorn Leesatapornwongsa、Xin Wang、Shishir Patil、Tianjun Zhang、Saehan Jo、Ishai Menache、Kontantina Mellou、Runlong Zhou、Feiran Jia、Hamed Khanpour、Hamid Palangi、Srinagesh Sharma、Julio Albinati Cortez、Amin Saied、Yuzhe Ma、Dujian Ding、Linyong Nan、Prateek Yadav、Shannon Shen、Ankur Mallick、Mark Encarnación、Lars Liden、Tianwei Yue、Julia Kiseleva、Anastasia Razdaibiedina 和 Luciano Del Corro 的讨论

与反馈。Qingyun Wu 感谢宾夕法尼亚州立大学信息科学与技术学院提供的资助和研究支持。

参考文献

[1] Vaibhav Adlakha, Parishad BehnamGhader, Xing Han Lu, Nicholas Meade, and Siva Reddy. Evaluating correctness and faithfulness of instruction-following models for question answering. arXiv preprint arXiv:2307.16877, 2023.

[2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. Guidelines for human-AI interaction. In Proceedings of the 2019 chi conference on human factors in computing systems, 2019.

[3] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016.

[4] AutoGPT. Documentation — Auto-GPT. https://docs.agpt.co/, 2023.

[5] BabyAGI. GitHub — BabyAGI. https://github.com/yoheinakajima/babyagi, 2023.

[6] Carrie J. Cai, Samantha Winter, David F. Steiner, Lauren Wilcox, and Michael Terry. "hello ai": Uncovering the onboarding needs of medical practitioners for human-AI collaborative decision-making. Proceedings of the ACM on Human-Computer Interaction, 2019.

[7] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. arXiv preprint arXiv:2305.17126, 2023.

[8] Chroma. Chromadb. https://github.com/chroma-core/chroma, 2023.

[9] Victor Dibia. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), Toronto, Canada, July 2023. Association for Computational Linguistics.

[10] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. Self-collaboration code generation via ChatGPT. arXiv preprint arXiv:2304.07590, 2023.

[11] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325, 2023.

[12] Atty Eleti, Jeff Harris, and Logan Kilpatrick. Function calling and other API updates. https://openai.com/blog/function-calling-and-other-api-updates, 2023.

[13] Guidance. Guidance. https://github.com/guidance-ai/guidance, 2023.

[14] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[15] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, et al. MetaGPT: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, 2023.

[16] Eric Horvitz. Principles of mixed-initiative user interfaces. In Proceedings of the SIGCHI conference on Human Factors in Computing Systems, 1999.

[17] HuggingFace. Transformers agent. https://huggingface.co/docs/transformers/transformers_age

nts, 2023.
[18] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. Language models can solve computer tasks. arXiv preprint arXiv:2303.17491, 2023.
[19] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. Transactions of the Association for Computational Linguistics, 2019.
[20] LangChain. Introduction — LangChain. https://python.langchain.com/en/latest/index.html, 2023.
[21] Mike Lewis, Denis Yarats, Yann N Dauphin, Devi Parikh, and Dhruv Batra. Deal or no deal? end-to-end learning for negotiation dialogues. arXiv preprint arXiv:1706.05125, 2017.
[22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in Neural Information Processing Systems, 2020.
[23] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. arXiv preprint arXiv:2307.03875, 2023a.
[24] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large scale language model society, 2023b.
[25] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate, 2023.
[26] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. arXiv preprint arXiv:1802.08802, 2018.
[27] Jerry Liu. LlamaIndex, November 2022. URL https://github.com/jerryjliu/llama_index.
[28] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.
[29] Roberto Navigli, Simone Conia, and Björn Ross. Biases in large language models: Origins, inventory and discussion. ACM Journal of Data and Information Quality, 2023.
[30] OpenAI. ChatGPT plugins. https://openai.com/blog/chatgpt-plugins, 2023.
[31] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. arXiv preprint arXiv:2304.03442, 2023.
[32] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Retrieval augmented code generation and summarization. arXiv preprint arXiv:2108.11601, 2021.
[33] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language

model connected with massive APIs. arXiv preprint arXiv:2305.15334, 2023.
[34] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 11 2019. URL https://arxiv.org/abs/1908.10084.
[35] Semantic-Kernel. Semantic Kernel. https://github.com/microsoft/semantic-kernel, 2023.
[36] Bokui Shen, Fei Xia, Chengshu Li, Roberto Martín-Martín, Linxi Fan, Guanzhi Wang, Claudia Pérez-D'Arpino, Shyamal Buch, Sanjana Srivastava, Lyne Tchapmi, et al. iGibson 1.0: A simulation environment for interactive tasks in large realistic scenes. In 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2021.
[37] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In International Conference on Machine Learning. PMLR, 2017.
[38] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In Proceedings of the International Conference on Learning Representations (ICLR), 2021. URL https://arxiv.org/abs/2010.03768.
[39] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. arXiv preprint arXiv:1708.04782, 2017.
[40] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. Flaml: A fast and lightweight AutoML library. Proceedings of Machine Learning and Systems, 2021.
[41] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023a.
[42] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. arXiv preprint arXiv:2308.11432, 2023b.
[43] Daniel S. Weld and Oren Etzioni. The first law of robotics (a call to arms). In AAAI Conference on Artificial Intelligence, 1994.
[44] Max Woolf. LangChain problem. https://minimaxir.com/2023/07/langchain-problem/, 2023.
[45] Yiran Wu, Feiran Jia, Shaokun Zhang, Qingyun Wu, Hangyu Li, Erkang Zhu, Yue Wang, Yin Tat Lee, Richard Peng, and Chi Wang. An empirical study on challenging math problem solving with GPT-4. arXiv preprint arXiv:2306.01337, 2023.
[46] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. arXiv preprint arXiv:2309.07864, 2023.
[47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022.

AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

Qingyun Wu[†], **Gagan Bansal**[*], **Jieyu Zhang**[±], **Yiran Wu**[†], **Beibin Li**[*]

Erkang Zhu[*], **Li Jiang**[*], **Xiaoyun Zhang**[*], **Shaokun Zhang**[†], **Jiale Liu**[∓]

Ahmed Awadallah[*], **Ryen W. White**[*], **Doug Burger**[*], **Chi Wang**[*1]

[*]Microsoft Research, [†]Pennsylvania State University

[±]University of Washington,[∓]Xidian University

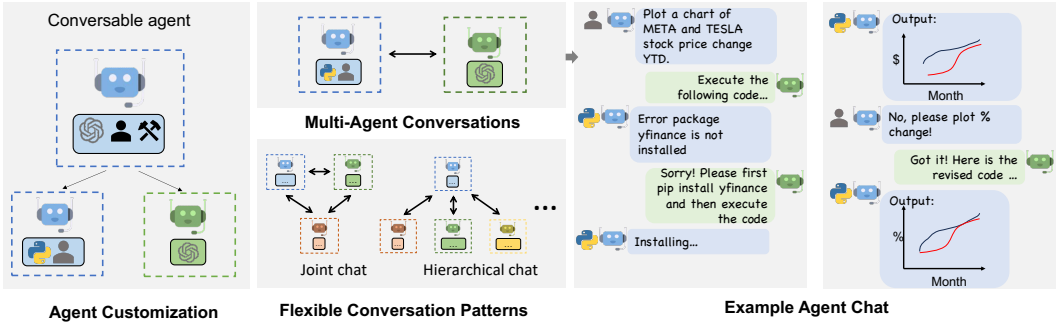


Figure 1: AutoGen enables diverse LLM-based applications using multi-agent conversations. (Left) AutoGen agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them. (Top-middle) Agents can converse to solve tasks. (Right) They can form a chat, potentially with humans in the loop. (Bottom-middle) The framework supports flexible conversation patterns.

Abstract

AutoGen[2] is an open-source framework that allows developers to build LLM applications via multiple *agents* that can converse with each other to accomplish tasks. AutoGen agents are customizable, *conversable*, and can operate in various modes that employ combinations of LLMs, human inputs, and tools. Using AutoGen, developers can also flexibly define agent interaction behaviors. Both natural language and computer code can be used to program flexible conversation patterns for different applications. AutoGen serves as a generic framework for building diverse applications of various complexities and LLM capacities. Empirical studies demonstrate the effectiveness of the framework in many example applications, with domains ranging from mathematics, coding, question answering, operations research, online decision-making, entertainment, etc.

[1]Corresponding author. Email: auto-gen@outlook.com

[2]https://github.com/microsoft/autogen

1 Introduction

Large language models (LLMs) are becoming a crucial building block in developing powerful *agents* that utilize LLMs for reasoning, tool usage, and adapting to new observations (Yao et al., 2022; Xi et al., 2023; Wang et al., 2023b) in many real-world tasks. Given the expanding tasks that could benefit from LLMs and the growing task complexity, an intuitive approach to scale up the power of agents is to use multiple agents that cooperate. Prior work suggests that multiple agents can help encourage divergent thinking (Liang et al., 2023), improve factuality and reasoning (Du et al., 2023), and provide validation (Wu et al., 2023). In light of the intuition and early evidence of promise, it is intriguing to ask the following question: *how* can we facilitate the development of LLM applications that could span a broad spectrum of domains and complexities based on the multi-agent approach?

Our insight is to use *multi-agent conversations* to achieve it. There are at least three reasons confirming its general feasibility and utility thanks to recent advances in LLMs: First, because chat-optimized LLMs (e.g., GPT-4) show the ability to incorporate feedback, LLM agents can cooperate through *conversations* with each other or human(s), e.g., a dialog where agents provide and seek reasoning, observations, critiques, and validation. Second, because a single LLM can exhibit a broad range of capabilities (especially when configured with the correct prompt and inference settings), conversations between differently configured agents can help combine these broad LLM capabilities in a modular and complementary manner. Third, LLMs have demonstrated ability to solve complex tasks when the tasks are broken into simpler subtasks. Multi-agent conversations can enable this partitioning and integration in an intuitive manner. How can we leverage the above insights and support different applications with the common requirement of coordinating multiple agents, potentially backed by LLMs, humans, or tools exhibiting different capacities? We desire a multi-agent conversation framework with generic abstraction and effective implementation that has the flexibility to satisfy different application needs. Achieving this requires addressing two critical questions: (1) How can we design individual agents that are capable, reusable, customizable, and effective in multi-agent collaboration? (2) How can we develop a straightforward, unified interface that can accommodate a wide range of agent conversation patterns? In practice, applications of varying complexities may need distinct sets of agents with specific capabilities, and may require different conversation patterns, such as single- or multi-turn dialogs, different human involvement modes, and static vs. dynamic conversation. Moreover, developers may prefer the flexibility to program agent interactions in natural language or code. Failing to adequately address these two questions would limit the framework's scope of applicability and generality.

While there is contemporaneous exploration of multi-agent approaches,[3] we present `AutoGen`, a generalized multi-agent conversation framework (Figure 1), based on the following new concepts.

1 **Customizable and conversable agents.** `AutoGen` uses a generic design of agents that can leverage LLMs, human inputs, tools, or a combination of them. The result is that developers can easily and quickly create agents with different roles (e.g., agents to write code, execute code, wire in human feedback, validate outputs, etc.) by selecting and configuring a subset of built-in capabilities. The agent's backend can also be readily extended to allow more custom behaviors. To make these agents suitable for multi-agent conversation, every agent is made *conversable* – they can receive, react, and respond to messages. When configured properly, an agent can hold multiple turns of conversations with other agents autonomously or solicit human inputs at certain rounds, enabling human agency and automation. The conversable agent design leverages the strong capability of the most advanced LLMs in taking feedback and making progress via chat and also allows combining capabilities of LLMs in a modular fashion. (Section 2.1)

2 **Conversation programming.** A fundamental insight of `AutoGen` is to simplify and unify complex LLM application workflows as multi-agent conversations. So `AutoGen` adopts a programming paradigm centered around these inter-agent conversations. We refer to this paradigm as *conversation programming*, which streamlines the development of intricate applications via two primary steps: (1) defining a set of conversable agents with specific capabilities and roles (as described above); (2) programming the interaction behavior between agents via conversation-centric *computation* and *control*. Both steps can be achieved via a fusion of natural and programming languages to build applications with a wide range of conversation patterns and agent behaviors. `AutoGen` provides ready-to-use implementations and also allows easy extension and experimentation for both steps. (Section 2.2)

[3] We refer to Appendix A for a detailed discussion.

AutoGen also provides a collection of multi-agent applications created using conversable agents and conversation programming. These applications demonstrate how AutoGen can easily support applications of various complexities and LLMs of various capabilities. Moreover, we perform both evaluation on benchmarks and a pilot study of new applications. The results show that AutoGen can help achieve outstanding performance on many tasks, and enable innovative ways of using LLMs, while reducing development effort. (Section 3 and Appendix D)

2 The AutoGen Framework

To reduce the effort required for developers to create complex LLM applications across various domains, a core design principle of AutoGen is to streamline and consolidate multi-agent workflows using multi-agent conversations. This approach also aims to maximize the reusability of implemented agents. This section introduces the two key concepts of AutoGen: conversable agents and conversation programming.

2.1 Conversable Agents

In AutoGen, a *conversable agent* is an entity with a specific role that can pass messages to send and receive information to and from other conversable agents, e.g., to start or continue a conversation. It maintains its internal context based on sent and received messages and can be configured to possess a set of capabilities, e.g., enabled by LLMs, tools, or human input, etc. The agents can act according to programmed behavior patterns described next.

Agent capabilities powered by LLMs, humans, and tools. Since an agent's capabilities directly influence how it processes and responds to messages, AutoGen allows flexibility to endow its agents with various capabilities. AutoGen supports many common composable capabilities for agents, including **1) LLMs.** LLM-backed agents exploit many capabilities of advanced LLMs such as role playing, implicit state inference and progress making conditioned on conversation history, providing feedback, adapting from feedback, and coding. These capabilities can be combined in different ways via novel prompting techniques[4] to increase an agent's skill and autonomy. AutoGen also offers enhanced LLM inference features such as result caching, error handling, message templating, etc., via an enhanced LLM inference layer. **2) Humans.** Human involvement is desired or even essential in many LLM applications. AutoGen lets a human participate in agent conversation via human-backed agents, which could solicit human inputs at certain rounds of a conversation depending on the agent configuration. The default *user proxy* agent allows *configurable* human involvement levels and patterns, e.g., frequency and conditions for requesting human input including the option for humans to skip providing input. **3) Tools.** Tool-backed agents have the capability to execute tools via code execution or function execution. For example, the default user proxy agent in AutoGen is able to execute code suggested by LLMs, or make LLM-suggested function calls.

Agent customization and cooperation. Based on application-specific needs, each agent can be configured to have a mix of basic back-end types to display complex behavior in multi-agent conversations. AutoGen allows easy creation of agents with specialized capabilities and roles by reusing or extending the built-in agents. The yellow-shaded area of Figure 2 provides a sketch of the built-in agents in AutoGen. The `ConversableAgent` class is the highest-level agent abstraction and, by default, can use LLMs, humans, and tools. The `AssistantAgent` and `UserProxyAgent` are two pre-configured `ConversableAgent` subclasses, each representing a common usage mode, i.e., acting as an AI assistant (backed by LLMs) and acting as a human proxy to solicit human input or execute code/function calls (backed by humans and/or tools).

In the example on the right-hand side of Figure 1, an LLM-backed assistant agent and a tool- and human-backed user proxy agent are deployed together to tackle a task. Here, the assistant agent generates a solution with the help of LLMs and passes the solution to the user proxy agent. Then, the user proxy agent solicits human inputs or executes the assistant's code and passes the results as feedback back to the assistant.

[4] Appendix C presents an example of such novel prompting techniques which empowers the default LLM-backed assistant agent in AutoGen to converse with other agents in multi-step problem solving.

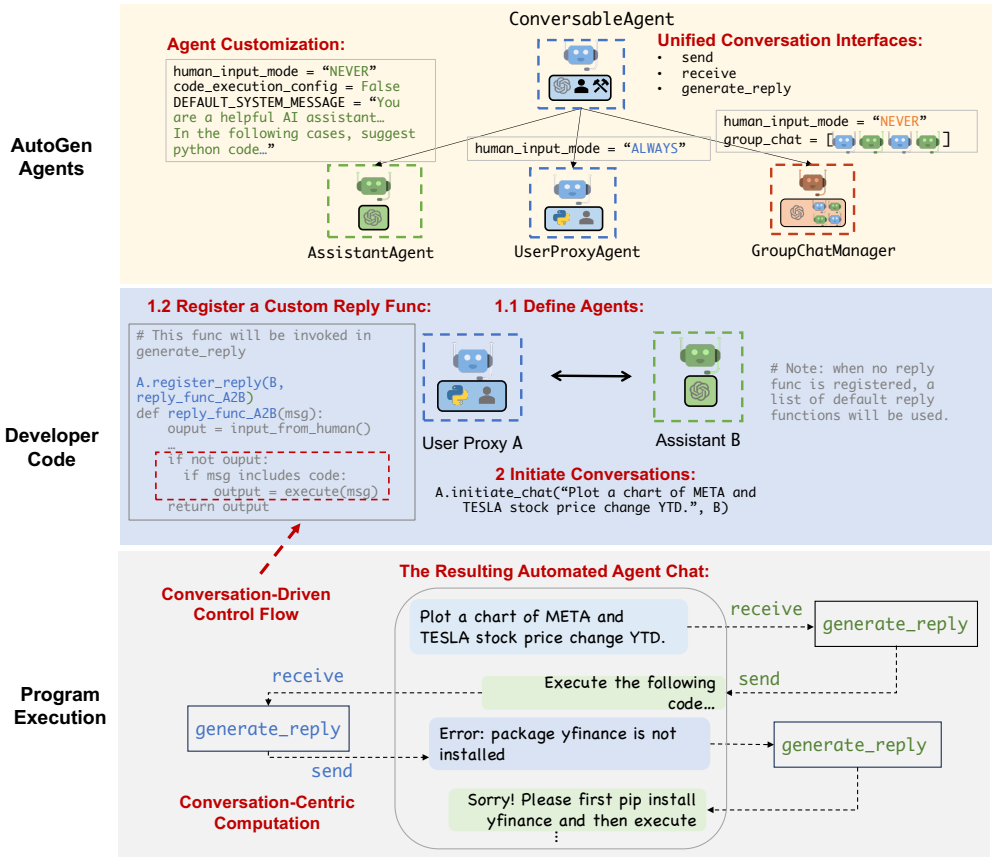


Figure 2: Illustration of how to use `AutoGen` to program a multi-agent conversation. The top sub-figure illustrates the built-in agents provided by `AutoGen`, which have unified conversation interfaces and can be customized. The middle sub-figure shows an example of using `AutoGen` to develop a two-agent system with a custom reply function. The bottom sub-figure illustrates the resulting automated agent chat from the two-agent system during program execution.

By allowing custom agents that can converse with each other, conversable agents in `AutoGen` serve as a useful building block. However, to develop applications where agents make meaningful progress on tasks, developers also need to be able to specify and mold these multi-agent conversations.

2.2 Conversation Programming

As a solution to the above problem, `AutoGen` utilizes *conversation programming*, a paradigm that considers two concepts: the first is *computation* – the actions agents take to compute their response in a multi-agent conversation. And the second is *control flow* – the sequence (or conditions) under which these computations happen. As we will show in the applications section, the ability to program these helps implement many flexible multi-agent conversation patterns. In `AutoGen`, these computations are conversation-centric. An agent takes actions relevant to the conversations it is involved in and its actions result in message passing for consequent conversations (unless a termination condition is satisfied). Similarly, control flow is conversation-driven – the participating agents' decisions on which agents to send messages to and the procedure of computation are functions of the inter-agent conversation. This paradigm helps one to reason intuitively about a complex workflow as agent action taking and conversation message-passing between agents.

Figure 2 provides a simple illustration. The bottom sub-figure shows how individual agents perform their role-specific, conversation-centric computations to generate responses (e.g., via LLM inference calls and code execution). The task progresses through conversations displayed in the dialog box. The middle sub-figure demonstrates a conversation-based control flow. When the assistant receives a message, the user proxy agent typically sends the human input as a reply. If there is no input, it executes any code in the assistant's message instead.

AutoGen features the following design patterns to facilitate conversation programming:

1. **Unified interfaces and auto-reply mechanisms for automated agent chat.** Agents in AutoGen have unified conversation interfaces for performing the corresponding conversation-centric computation, including a `send/receive` function for sending/receiving messages and a `generate_reply` function for taking actions and generating a response based on the received message. AutoGen also introduces and by default adopts an **agent auto-reply** mechanism to realize conversation-driven control: Once an agent receives a message from another agent, it automatically invokes `generate_reply` and sends the reply back to the sender unless a termination condition is satisfied. AutoGen provides built-in reply functions based on LLM inference, code or function execution, or human input. One can also register custom reply functions to customize the behavior pattern of an agent, e.g., chatting with another agent before replying to the sender agent. Under this mechanism, once the reply functions are registered, and the conversation is initialized, the conversation flow is naturally induced, and thus the agent conversation proceeds naturally without any extra control plane, i.e., a special module that controls the conversation flow. For example, with the developer code in the blue-shaded area (marked "Developer Code") of Figure 2, one can readily trigger the conversation among the agents, and the conversation would proceed automatically, as shown in the dialog box in the grey shaded area (marked "Program Execution") of Figure 2. The auto-reply mechanism provides a decentralized, modular, and unified way to define the workflow.
2. **Control by fusion of programming and natural language.** AutoGen allows the usage of programming and natural language in various control flow management patterns: 1) **Natural-language control via LLMs.** In AutoGen, one can control the conversation flow by prompting the LLM-backed agents with natural language. For instance, the default system message of the built-in `AssistantAgent` in AutoGen uses natural language to instruct the agent to fix errors and generate code again if the previous result indicates there are errors. It also guides the agent to confine the LLM output to certain structures, making it easier for other tool-backed agents to consume. For example, instructing the agent to reply with "TERMINATE" when all tasks are completed to terminate the program. More concrete examples of natural language controls can be found in Appendix C. 2) **Programming-language control.** In AutoGen, Python code can be used to specify the termination condition, human input mode, and tool execution logic, e.g., the max number of auto replies. One can also register programmed auto-reply functions to control the conversation flow with Python code, as shown in the code block identified as "Conversation-Driven Control Flow" in Figure 2. 3) **Control transition between natural and programming language.** AutoGen also supports flexible control transition between natural and programming language. One can achieve transition from code to natural-language control by invoking an LLM inference containing certain control logic in a customized reply function; or transition from natural language to code control via LLM-proposed function calls (Eleti et al., 2023).

In the conversation programming paradigm, one can realize multi-agent conversations of diverse patterns. In addition to static conversation with predefined flow, AutoGen also supports dynamic conversation flows with multiple agents. AutoGen provides two general ways to achieve this: 1) Customized `generate_reply` function: within the customized `generate_reply` function, one agent can hold the current conversation while invoking conversations with other agents depending on the content of the current message and context. 2) Function calls: In this approach, LLM decides whether or not to call a particular function depending on the conversation status. By messaging additional agents in the called functions, the LLM can drive dynamic multi-agent conversation. In addition, AutoGen supports more complex dynamic group chat via built-in `GroupChatManager`, which can dynamically select the next speaker and then broadcast its response to other agents. We elaborate on this feature and its application in Section 3. We provide implemented working systems to showcase all these different patterns, with some of them visualized in Figure 3.

3 Applications of AutoGen

We demonstrate six applications using AutoGen (see Figure 3) to illustrate its potential in simplifying the development of high-performance multi-agent applications. These applications are selected based on their real-world relevance (A1, A2, A4, A5, A6), problem difficulty and solving capabilities enabled by AutoGen (A1, A2, A3, A4), and innovative potential (A5, A6). Together, these criteria showcase AutoGen's role in advancing the LLM-application landscape.

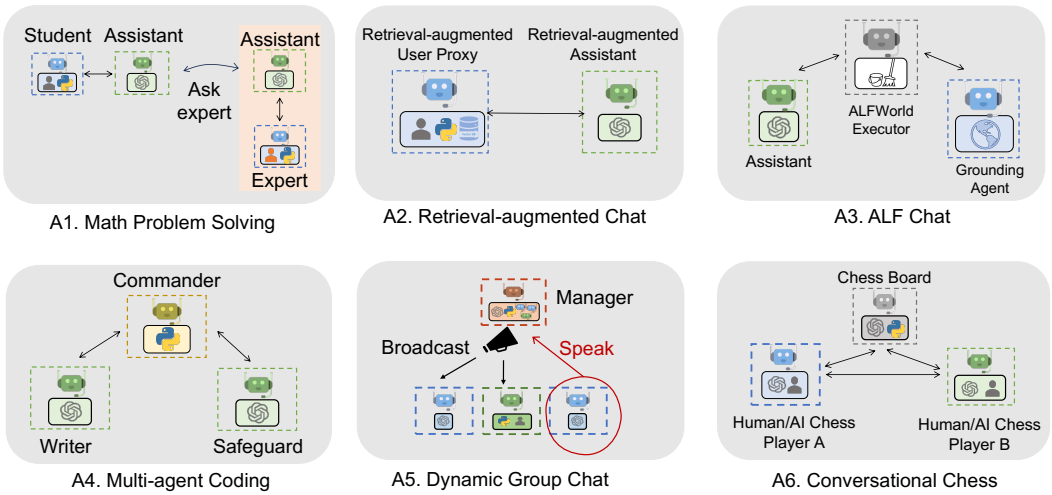


Figure 3: Six examples of diverse applications built using `AutoGen`. Their conversation patterns show `AutoGen`'s flexibility and power.

A1: Math Problem Solving

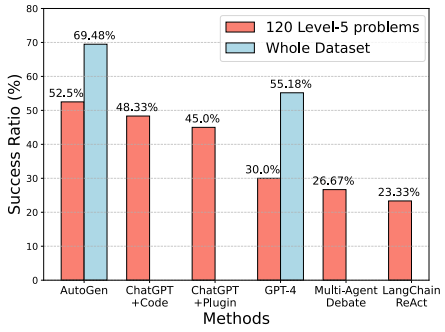
Mathematics is a foundational discipline and the promise of leveraging LLMs to assist with math problem solving opens up a new plethora of applications and avenues for exploration, including personalized AI tutoring, AI research assistance, etc. This section demonstrates how `AutoGen` can help develop LLM applications for math problem solving, showcasing strong performance and flexibility in supporting various problem-solving paradigms.

(**Scenario 1**) We are able to build a system for autonomous math problem solving by directly reusing two built-in agents from `AutoGen`. We evaluate our system and several alternative approaches, including open-source methods such as Multi-Agent Debate (Liang et al., 2023), LangChain ReAct (LangChain, 2023), vanilla GPT-4, and commercial products ChatGPT + Code Interpreter, and ChatGPT + Plugin (Wolfram Alpha), on the MATH (Hendrycks et al., 2021) dataset and summarize the results in Figure 4a. We perform evaluations over 120 randomly selected level-5 problems and on the entire[5] test dataset from MATH. The results show that the built-in agents from `AutoGen` already yield better performance out of the box compared to the alternative approaches, even including the commercial ones. (**Scenario 2**) We also showcase a human-in-the-loop problem-solving process with the help of `AutoGen`. To incorporate human feedback with `AutoGen`, one only needs to set `human_input_mode='ALWAYS'` in the `UserProxyAgent` of the system in scenario 1. We demonstrate that this system can effectively incorporate human inputs to solve challenging problems that cannot be solved without humans. (**Scenario 3**) We further demonstrate a novel scenario where *multiple* human users can participate in the conversations during the problem-solving process. Our experiments and case studies for these scenarios show that AutoGen enables better performance or new experience compared to other solutions we experimented with. Due to the page limit, details of the evaluation, including case studies in three scenarios are in Appendix D.

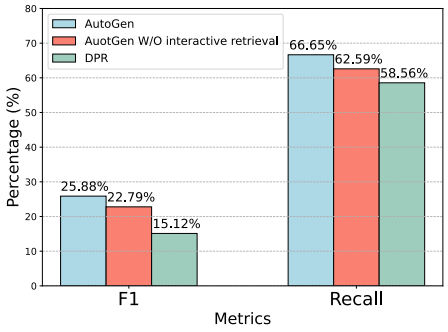
A2: Retrieval-Augmented Code Generation and Question Answering

Retrieval augmentation has emerged as a practical and effective approach for mitigating the intrinsic limitations of LLMs by incorporating external documents. In this section, we employ `AutoGen` to build a Retrieval-Augmented Generation (RAG) system (Lewis et al., 2020; Parvez et al., 2021) named Retrieval-augmented Chat. The system consists of two agents: a Retrieval-augmented User Proxy agent and a Retrieval-augmented Assistant agent, both of which are extended from built-in agents from `AutoGen`. The Retrieval-augmented User Proxy includes a vector database (Chroma,

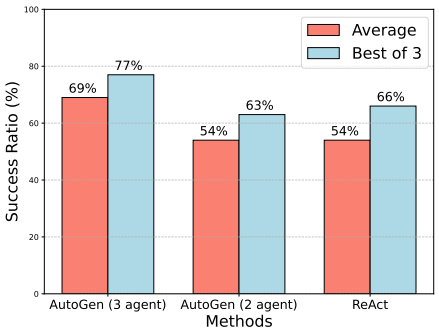
[5]We did not evaluate ChatGPT on the whole dataset since it requires substantial manual effort and is restricted by its hourly message-number limitation. Multi-agent debate and LangChain ReAct were also not evaluated since they underperformed vanilla GPT-4 on the smaller test set.



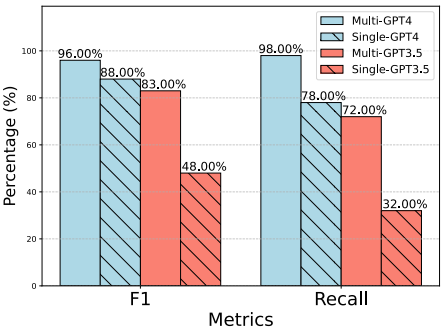
(a) A1: Performance on MATH (w/ GPT-4).



(b) A2: Q&A tasks (w/ GPT-3.5).



(c) A3: Performance on ALFWorld.



(d) A4: Performance on OptiGuide.

Figure 4: Performance on four applications A1-A4. (a) shows that `AutoGen` agents can be used out of the box to achieve the most competitive performance on math problem solving tasks; (b) shows that `AutoGen` can be used to realize effective retrieval augmentation and realize a novel interactive retrieval feature to boost performance on Q&A tasks; (c) shows that `AutoGen` can be used to introduce a three-agent system with a grounding agent to improve performance on ALFWorld; (d) shows that a multi-agent design is helpful in boosting performance in coding tasks that need safeguards.

2023) with SentenceTransformers (Reimers & Gurevych, 2019) as the context retriever. A detailed workflow description of the Retrieval-augmented Chat is provided in Appendix D.

We evaluate Retrieval-augmented Chat in both question-answering and code-generation scenarios. (**Scenario 1**) We first perform an evaluation regarding natural question answering on the Natural Questions dataset (Kwiatkowski et al., 2019) and report results in Figure 4b. In this evaluation, we compare our system with DPR (Dense Passage Retrieval) following an existing evaluation[6] practice (Adlakha et al., 2023). Leveraging the conversational design and natural-language control, `AutoGen` introduces a novel *interactive retrieval* feature in this application: whenever the retrieved context does not contain the information, instead of terminating, the LLM-based assistant would reply "*Sorry, I cannot find any information about... UPDATE CONTEXT.*" which will invoke more retrieval attempts. We conduct an ablation study in which we prompt the assistant agent to say *"I don't know"* instead of *"UPDATE CONTEXT."* in cases where relevant information is not found, and report results in Figure 4b. The results show that the interactive retrieval mechanism indeed plays a non-trivial role in the process. We give a concrete example and results using this appealing feature in Appendix D. (**Scenario 2**) We further demonstrate how Retrieval-augmented Chat aids in generating code based on a given codebase that contains code not included in GPT-4's training data. Evaluation and demonstration details for both scenarios are included in Appendix D.

[6]The results of DPR with GPT-3.5 shown in Figure 4b are from (Adlakha et al., 2023). We use GPT-3.5 as a shorthand for GPT-3.5-turbo.

A3: Decision Making in Text World Environments

In this subsection, we demonstrate how `AutoGen` can be used to develop effective applications that involve interactive or online decision making. We perform the study using the ALFWorld (Shridhar et al., 2021) benchmark, which includes a diverse collection of synthetic language-based interactive decision-making tasks in household environments.

With `AutoGen`, we implemented a two-agent system to solve tasks from ALFWorld. It consists of an LLM-backed assistant agent responsible for suggesting plans to conduct a task and an executor agent responsible for executing actions in the ALFWorld environments. This system integrates ReAct prompting (Yao et al., 2022), and is able to achieve similar performance. A common challenge encountered in both ReAct and the `AutoGen`-based two-agent system is their occasional inability to leverage basic commonsense knowledge about the physical world. This deficiency can lead to the system getting stuck in a loop due to repetitive errors. Fortunately, the modular design of `AutoGen` allows us to address this issue effectively: With `AutoGen`, we are able to introduce a grounding agent, which supplies crucial commonsense knowledge–such as *"You must find and take the object before you can examine it. You must go to where the target object is before you can use it."*–whenever the system exhibits early signs of recurring errors. It significantly enhances the system's ability to avoid getting entangled in error loops. We compare the task-solving performance of the two variants of our system with GPT-3.5-turbo and ReAct[7] on the 134 unseen tasks from ALFWorld and report results in Figure 4c. The results show that introducing a grounding agent could bring in a 15% performance gain on average. Upon examining the systems' outputs, we observe that the grounding agent, by delivering background commonsense knowledge at the right junctures, significantly mitigated the tendency of the system to persist with a flawed plan, thereby avoiding the creation of error loops. For an example trajectory comparing the systems see Appendix D, Figure 10.

A4: Multi-Agent Coding

In this subsection, we use `AutoGen` to build a multi-agent coding system based on OptiGuide (Li et al., 2023a), a system that excels at writing code to interpret optimization solutions and answer user questions, such as exploring the implications of changing a supply-chain decision or understanding why the optimizer made a particular choice. The second sub-figure of Figure 3 shows the `AutoGen`-based implementation. The workflow is as follows: the end user sends questions, such as "*What if we prohibit shipping from supplier 1 to roastery 2?*" to the Commander agent. The Commander coordinates with two assistant agents, including the Writer and the Safeguard, to answer the question. The Writer will craft code and send the code to the Commander. After receiving the code, the Commander checks the code safety with the Safeguard; if cleared, the Commander will use external tools (e.g., Python) to execute the code, and request the Writer to interpret the execution results. For instance, the writer may say "*if we prohibit shipping from supplier 1 to roastery 2, the total cost would increase by 10.5%.*" The Commander then provides this concluding answer to the end user. If, at a particular step, there is an exception, e.g., security red flag raised by Safeguard, the Commander redirects the issue back to the Writer with debugging information. The process might be repeated multiple times until the user's question is answered or timed-out.

With `AutoGen` the core workflow code for OptiGuide was reduced from over 430 lines to 100 lines, leading to significant productivity improvement. We provide a detailed comparison of user experience with ChatGPT+Code Interpreter and `AutoGen`-based OptiGuide in Appendix D, where we show that `AutoGen`-based OptiGuide could save around 3x of user's time and reduce user interactions by 3 - 5 times on average. We also conduct an ablation showing that multi-agent abstraction is necessary. Specifically, we construct a single-agent approach where a single agent conducts both the code-writing and safeguard processes. We tested the single- and multi-agent approaches on a dataset of 100 coding tasks, which is crafted to include equal numbers of safe and unsafe tasks. Evaluation results as reported in Figure 4d show that the multi-agent design boosts the F-1 score in identifying unsafe code by 8% (with GPT-4) and 35% (with GPT-3.5-turbo).

[7] Results of ReAct are obtained by directly running its official code with default settings. The code uses `text-davinci-003` as backend LM and does not support GPT-3.5-turbo or GPT-4.

A5: Dynamic Group Chat

AutoGen provides native support for a *dynamic group chat* communication pattern, in which participating agents share the same context and converse with the others in a dynamic manner instead of following a pre-defined order. Dynamic group chat relies on ongoing conversations to guide the flow of interaction among agents. These make dynamic group chat ideal for situations where collaboration without strict communication order is beneficial. In AutoGen, the GroupChatManager class serves as the conductor of conversation among agents and repeats the following three steps: dynamically selecting a speaker, collecting responses from the selected speaker, and broadcasting the message (Figure 3-A5). For the dynamic speaker-selection component, we use a role-play style prompt. Through a pilot study on 12 manually crafted complex tasks, we observed that compared to a prompt that is purely based on the task, utilizing a role-play prompt often leads to more effective consideration of both conversation context and role alignment during the problem-solving and speaker-selection process. Consequently, this leads to a higher success rate and fewer LLM calls. We include detailed results in Appendix D.

A6: Conversational Chess

Using AutoGen, we developed Conversational Chess, a natural language interface game shown in the last sub-figure of Figure 3. It features built-in agents for players, which can be human or LLM, and a third-party board agent to provide information and validate moves based on standard rules.

With AutoGen, we enabled two essential features: (1) Natural, flexible, and engaging game dynamics, enabled by the customizable agent design in AutoGen. Conversational Chess supports a range of game-play patterns, including AI-AI, AI-human, and human-human, with seamless switching between these modes during a single game. An illustrative example of these entertaining game dynamics can be found in Figure 15, Appendix D. (2) Grounding, which is a crucial aspect to maintain game integrity. During gameplay, the board agent checks each proposed move for legality; if a move is invalid, the agent responds with an error, prompting the player agent to re-propose a legal move before continuing. This process ensures that only valid moves are played and helps maintain a consistent gaming experience. As an ablation study, we removed the board agent and instead only relied on a relevant prompt *"you should make sure both you and the opponent are making legal moves"* to ground their move. The results highlighted that without the board agent, illegitimate moves caused game disruptions. The modular design offered flexibility, allowing swift adjustments to the board agent in response to evolving game rules or varying chess rule variants. A comprehensive demonstration of this ablation study is in Appendix D.

4 Discussion

We introduced an open-source library, AutoGen, that incorporates the paradigms of conversable agents and conversation programming. This library utilizes capable agents that are well-suited for multi-agent cooperation. It features a unified conversation interface among the agents, along with an auto-reply mechanisms, which help establish an agent-interaction interface that capitalizes on the strengths of chat-optimized LLMs with broad capabilities while accommodating a wide range of applications. AutoGen serves as a general framework for creating and experimenting with multi-agent systems that can easily fulfill various practical requirements, such as reusing, customizing, and extending existing agents, as well as programming conversations between them.

Our experiments, as detailed in Section 3, demonstrate that this approach offers numerous benefits. The adoption of AutoGen has resulted in improved performance (over state-of-the-art approaches), reduced development code, and decreased manual burden for existing applications. It offers flexibility to developers, as demonstrated in A1 (scenario 3), A5, and A6, where AutoGen enables multi-agent chats to follow a dynamic pattern rather than fixed back-and-forth interactions. It allows humans to engage in activities alongside multiple AI agents in a conversational manner. Despite the complexity of these applications (most involving more than two agents or dynamic multi-turn agent cooperation), the implementation based on AutoGen remains straightforward. Dividing tasks among separate agents promotes modularity. Furthermore, since each agent can be developed, tested, and maintained separately, this approach simplifies overall development and code management.

Although this work is still in its early experimental stages, it paves the way for numerous future directions and research opportunities. For instance, we can explore effective integration of existing agent implementations into our multi-agent framework and investigate the optimal balance between automation and human control in multi-agent workflows. As we further develop and refine `AutoGen`, we aim to investigate which strategies, such as agent topology and conversation patterns, lead to the most effective multi-agent conversations while optimizing the overall efficiency, among other factors. While increasing the number of agents and other degrees of freedom presents opportunities for tackling more complex problems, it may also introduce new safety challenges that require additional studies and careful consideration.

We provide more discussion in Appendix B, including guidelines for using `AutoGen` and direction of future work. We hope `AutoGen` will help improve many LLM applications in terms of speed of development, ease of experimentation, and overall effectiveness and safety. We actively welcome contributions from the broader community.

Ethics statement

There are several potential ethical considerations that could arise from the development and use of the `AutoGen` framework.

- Privacy and Data Protection: The framework allows for human participation in conversations between agents. It is important to ensure that user data and conversations are protected, and that developers use appropriate measures to safeguard privacy.
- Bias and Fairness: LLMs have been shown to exhibit biases present in their training data (Navigli et al., 2023). When using LLMs in the `AutoGen` framework, it is crucial to address and mitigate any biases that may arise in the conversations between agents. Developers should be aware of potential biases and take steps to ensure fairness and inclusivity.
- Accountability and Transparency: As discussed in the future work section, as the framework involves multiple agents conversing and cooperating, it is important to establish clear accountability and transparency mechanisms. Users should be able to understand and trace the decision-making process of the agents involved in order to ensure accountability and address any potential issues or biases.
- Trust and Reliance: `AutoGen` leverages human understanding and intelligence while providing automation through conversations between agents. It is important to consider the impact of this interaction on user experience, trust, and reliance on AI systems. Clear communication and user education about the capabilities and limitations of the system will be essential (Cai et al., 2019).
- Unintended Consequences: As discussed before, the use of multi-agent conversations and automation in complex tasks may have unintended consequences. In particular, allowing LLM agents to make changes in external environments through code execution or function calls, such as installing packages, could be risky. Developers should carefully consider the potential risks and ensure that appropriate safeguards are in place to prevent harm or negative outcomes.

Acknowledgements

The work presented in this report was made possible through discussions and feedback from Peter Lee, Johannes Gehrke, Eric Horvitz, Steven Lucco, Umesh Madan, Robin Moeur, Piali Choudhury, Saleema Amershi, Adam Fourney, Victor Dibia, Guoqing Zheng, Corby Rosset, Ricky Loynd, Ece Kamar, Rafah Hosn, John Langford, Ida Momennejad, Brian Krabach, Taylor Webb, Shanka Subhra Mondal, Wei-ge Chen, Robert Gruen, Yinan Li, Yue Wang, Suman Nath, Tanakorn Leesatapornwongsa, Xin Wang, Shishir Patil, Tianjun Zhang, Saehan Jo, Ishai Menache, Kontantina Mellou, Runlong Zhou, Feiran Jia, Hamed Khanpour, Hamid Palangi, Srinagesh Sharma, Julio Albinati Cortez, Amin Saied, Yuzhe Ma, Dujian Ding, Linyong Nan, Prateek Yadav, Shannon Shen, Ankur Mallick, Mark Encarnación, Lars Liden, Tianwei Yue, Julia Kiseleva, Anastasia Razdaibiedina, and Luciano Del Corro. Qingyun Wu would like to acknowledge the funding and research support from the College of Information Science and Technology at Penn State University.

References

Vaibhav Adlakha, Parishad BehnamGhader, Xing Han Lu, Nicholas Meade, and Siva Reddy. Evaluating correctness and faithfulness of instruction-following models for question answering. *arXiv preprint arXiv:2307.16877*, 2023.

Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. Guidelines for human-ai interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*, 2019.

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety, 2016.

AutoGPT. Documentation — auto-gpt. `https://docs.agpt.co/`, 2023.

BabyAGI. Github — babyagi. `https://github.com/yoheinakajima/babyagi`, 2023.

Carrie J. Cai, Samantha Winter, David F. Steiner, Lauren Wilcox, and Michael Terry. "hello ai": Uncovering the onboarding needs of medical practitioners for human-ai collaborative decision-making. *Proceedings of the ACM on Human-Computer Interaction*, 2019.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.

Chroma. Chromadb. `https://github.com/chroma-core/chroma`, 2023.

Victor Dibia. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Toronto, Canada, July 2023. Association for Computational Linguistics.

Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. Self-collaboration code generation via chatgpt. *arXiv preprint arXiv:2304.07590*, 2023.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.

Atty Eleti, Jeff Harris, and Logan Kilpatrick. Function calling and other api updates. `https://openai.com/blog/function-calling-and-other-api-updates`, 2023.

Guidance. Guidance. `https://github.com/guidance-ai/guidance`, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*, 1999.

HuggingFace. Transformers agent. `https://huggingface.co/docs/transformers/transformers_agents`, 2023.

Geunwoo Kim, Pierre Baldi, and Stephen McAleer. Language models can solve computer tasks. *arXiv preprint arXiv:2303.17491*, 2023.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 2019.

LangChain. Introduction — langchain. https://python.langchain.com/en/latest/index.html, 2023.

Mike Lewis, Denis Yarats, Yann N Dauphin, Devi Parikh, and Dhruv Batra. Deal or no deal? end-to-end learning for negotiation dialogues. *arXiv preprint arXiv:1706.05125*, 2017.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 2020.

Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023a.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large scale language model society, 2023b.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate, 2023.

Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. *arXiv preprint arXiv:1802.08802*, 2018.

Jerry Liu. LlamaIndex, November 2022. URL https://github.com/jerryjliu/llama_index.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Roberto Navigli, Simone Conia, and Björn Ross. Biases in large language models: Origins, inventory and discussion. *ACM Journal of Data and Information Quality*, 2023.

OpenAI. ChatGPT plugins. https://openai.com/blog/chatgpt-plugins, 2023.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Retrieval augmented code generation and summarization. *arXiv preprint arXiv:2108.11601*, 2021.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL https://arxiv.org/abs/1908.10084.

Semantic-Kernel. Semantic kernel. https://github.com/microsoft/semantic-kernel, 2023.

Bokui Shen, Fei Xia, Chengshu Li, Roberto Martín-Martín, Linxi Fan, Guanzhi Wang, Claudia Pérez-D'Arpino, Shyamal Buch, Sanjana Srivastava, Lyne Tchapmi, et al. igibson 1.0: A simulation environment for interactive tasks in large realistic scenes. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *International Conference on Machine Learning*. PMLR, 2017.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2010.03768.

Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017.

Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. Flaml: A fast and lightweight automl library. *Proceedings of Machine Learning and Systems*, 2021.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023b.

Daniel S. Weld and Oren Etzioni. The first law of robotics (a call to arms). In *AAAI Conference on Artificial Intelligence*, 1994.

Max Woolf. Langchain problem. https://minimaxir.com/2023/07/langchain-problem/, 2023.

Yiran Wu, Feiran Jia, Shaokun Zhang, Qingyun Wu, Hangyu Li, Erkang Zhu, Yue Wang, Yin Tat Lee, Richard Peng, and Chi Wang. An empirical study on challenging math problem solving with gpt-4. *arXiv preprint arXiv:2306.01337*, 2023.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.